

Titanic Dataset — Exploratory Data Analysis

Introduction

The Titanic dataset contains information about passengers aboard the RMS Titanic.

The main objective of this exploratory data analysis (EDA) is to understand the dataset, identify patterns and relationships between passenger characteristics and survival, detect missing values and outliers, and perform feature engineering to create useful variables for further analysis.

Objectives

- Understand the structure and characteristics of the dataset
- Identify missing values
- Analyze numerical and categorical features
- Study relationships between passenger features and survival
- Detect potential outliers
- Perform feature engineering
- Summarize important findings from the analysis

Table of Contents

1. [Dataset Overview](#)
2. [Understanding the Features](#)
3. [Univariate Analysis](#)
4. [Bivariate Analysis](#)
5. [Feature Engineering](#)
6. [Key Findings](#)
7. [Next Steps](#)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

1. Dataset Overview

Let's begin by loading the dataset and examining its basic structure.

We will check:

- First few rows
- Number of rows and columns
- Column names
- Data types
- Basic statistical information
- Missing values

```
titanic = pd.read_csv('/kaggle/input/datasets/shuofxz/titanic-machine-learning-from-disaster/train.csv')
titanic.head()

titanic.shape
```

Dataset Dimensions

The dataset contains 891 observations and 12 columns.

Each row represents a passenger, while each column contains information about that passenger.

2. Understanding the Features

- PassengerId → Unique identifier for each passenger
- Survived → Survival status (0 = No, 1 = Yes) = Pclass → Passenger class (1st, 2nd, 3rd)
- Name → Passenger's name
- Sex → Passenger's gender
- Age → Passenger's age
- SibSp → Number of siblings/spouses aboard
- Parch → Number of parents/children aboard
- Ticket → Ticket number
- Fare → Passenger/ticket fare
- Cabin → Cabin number
- Embarked → Port of embarkation

Understanding these variables will help us interpret the patterns discovered during EDA.

Column Types

- **Numerical** - Age, Fare, PassengerId
- **Categorical** - Survived, Pclass, Sex, SibSp, Parch, Embarked
- **Mixed** - Name, Ticket, Cabin

3. Univariate Analysis

Univariate analysis focuses on analyzing each feature in the dataset independently.

- **Distribution analysis:** The distribution of each feature is examined to identify its shape, central tendency, and dispersion.

- **Identifying potential issues:** Univariate analysis helps in identifying potential problems with the data such as outliers, skewness, and missing values

Steps of doing Univariate Analysis on Numerical columns

- **Descriptive Statistics :** Compute basic summary statistics for the column, such as mean, median, mode, standard deviation, range, and quartiles. These statistics give a general understanding of the distribution of the data and can help identify skewness or outliers.
- **Visualizations :** Create visualizations to explore the distribution of the data. Some common visualizations for numerical data include histograms, box plots, and density plots. These visualizations provide a visual representation of the distribution of the data and can help identify skewness and outliers.
- **Identifying Outliers :** Identify and examine any outliers in the data. Outliers can be identified using visualizations. It is important to determine whether the outliers are due to measurement errors, data entry errors, or legitimate differences in the data, and to decide whether to include or exclude them from the analysis.
- **Skewness :** Check for skewness in the data and consider transforming the data or using robust statistical methods that are less sensitive to skewness, if necessary.
- **Conclusion :** Summarize the findings of the EDA and make decisions about how to proceed with further analysis.

3.1 Numerical Features

Age

Conclusions

- Age ranges from approximately 0.4 to 80 years.
- Around 20% of the Age values are missing.
- The distribution is broadly concentrated around young and middle-aged passengers.

```
titanic['Age'].describe()
```

```
titanic['Age'].plot(kind='hist', bins=20);
```

```
sns.kdeplot(data=titanic, x='Age');
```

```
titanic['Age'].plot(kind='box');
```

```
# check for outliers and print them
```

```
titanic[titanic['Age'] > 65]
```

```
(titanic['Age'].isnull().sum() / len(titanic['Age'])) * 100
```

Fare

Conclusions

- Data is highly positively skewed.
- The Fare column represents the fare associated with the ticket. When multiple passengers share the same ticket, this amount may represent the fare for the group rather than the amount attributable to one passenger.
- To obtain an approximate fare per passenger, we divide the ticket fare by the number of passengers associated with the ticket.

```
titanic['Fare'].describe()

titanic['Fare'].plot(kind='hist', bins=30);

sns.kdeplot(data=titanic, x='Fare', fill=True);

titanic['Fare'].skew()

titanic['Fare'].plot(kind='box');

# check the outliers
titanic[titanic['Fare'] > 250]

titanic['Fare'].isnull().sum()
```

Steps of doing Univariate Analysis on Categorical columns

Descriptive Statistics : Compute the frequency distribution of the categories in the column. This will give a general understanding of the distribution of the categories and their relative frequencies.

Visualizations : Create visualizations to explore the distribution of the categories. Some common visualizations for categorical data include count plots and pie charts. These visualizations provide a visual representation of the distribution of the categories and can help identify any patterns or anomalies in the data.

Missing Values : Check for missing values in the data and decide how to handle them. Missing values can be imputed or excluded from the analysis, depending on the research question and the data set.

Conclusion : Summarize the findings of the EDA and make decisions about how to proceed with further analysis.

3.2 Categorical Features

Survived

Conclusion

- Approximately 61.6% of passengers in the dataset did not survive, while 38.4% survived.

```
titanic['Survived'].value_counts()

titanic['Survived'].value_counts().plot(kind='bar');
```

```
titanic['Survived'].value_counts().plot(kind='pie', autopct='%0.1f%%',  
    labels=['Died', 'Survived'], ylabel = 'Survived Vs Died  
    Percentage');
```

```
titanic['Survived'].isnull().sum()
```

Pclass

Conclusion

- The number of passengers in Pclass 1 is 216.
- The number of passengers in Pclass 2 is 184.
- The number of passengers in Pclass 3 is 491.

```
titanic['Pclass'].value_counts()
```

```
titanic['Pclass'].value_counts().plot(kind='bar');
```

```
titanic['Pclass'].value_counts().plot(kind='pie', autopct='%0.1f%%');
```

```
titanic['Survived'].isnull().sum()
```

Sex

Conclusion

- There are 64.8% Males.
- There are 35.2% Females.

```
titanic['Sex'].value_counts()
```

```
titanic['Sex'].value_counts().plot(kind='bar');
```

```
titanic['Sex'].value_counts().plot(kind='pie', autopct='%0.1f%%');
```

```
titanic['Sex'].isnull().sum()
```

SibSp

Conclusion

- There are 608 people traveling alone.

```
titanic['SibSp'].value_counts()
```

```
titanic['SibSp'].value_counts().plot(kind='bar');
```

```
titanic['SibSp'].isnull().sum()
```

Parch

Conclusion

- There are 678 travelers traveling without parents.

- Parch and SibSp cols can be merged to form a new col call family_size.
- Create new col called is_alone.

```
titanic['Parch'].value_counts()
```

```
titanic['Parch'].value_counts().plot(kind='bar');
```

```
titanic['Parch'].isnull().sum()
```

Embarked

Conclusion

- There are 2 missing values.
- There are 72.4% travelers from the 'S' and 18.9% from 'C' and 8.7% from the 'Q'.

```
titanic['Embarked'].value_counts()
```

```
titanic['Embarked'].value_counts().plot(kind='pie', autopct='%0.1f%%');
```

```
titanic['Embarked'].isnull().sum()
```

```
# check the null cols
```

```
titanic[titanic['Embarked'] == None]
```

4. Bivariate Analysis

Bivariate analysis examines the relationship between two variables.

Since Survived is our target variable, we will focus on understanding how different passenger characteristics are associated with survival.

We will investigate relationships between:

- Survival and passenger class
- Survival and sex
- Survival and embarkation port
- Survival and age
- Other engineered features and survival

Steps in Bivariate Analysis

- Select Two Cols
- Understand type of relationship
 1. Numerical - Numerical → Plots graph like scatterplot, regression plot, 2D histplot, 2D kdeplot.
 2. Numerical - Categorical → Plots graph like barplot, kdeplot, violinplot, also scatter plot.
 3. Categorical - Categorical → Plots graph like heatmap, stacked barplot, treemaps.

- Conclusions

```
titanic.head()
```

4.1 Survival vs Passenger Class

Conclusions

- In Pclass 1 63.0% peoples are survived.
- In Pclass 2 47.3% peoples are survived.
- In Pclass 3 24.2% peoples are survived.
- So here we clear see this in the PClass 3 have highest survived chance and in the PClass 1 have lowest survived chance.

```
pd.crosstab(titanic['Survived'], titanic['Pclass'])

round(pd.crosstab(titanic['Survived'], titanic['Pclass'],
                 normalize='columns') * 100, 2)

round(pd.crosstab(titanic['Survived'], titanic['Pclass'],
                 normalize='columns') * 100, 2).plot(kind='pie', subplots=True,
                 autopct='%0.1f%%', figsize=(15,7), labels=['Died', 'Survived']);

plt.figure(figsize=(5,4))
sns.heatmap(round(pd.crosstab(titanic['Survived'], titanic['Pclass'],
                 normalize='columns') * 100, 2));
```

4.2 Survival vs Sex

Conclusions

- There are 74.2% females are survived and only 25.8% are died.
- There are 18.9% males are survived and only 81.1% are died.
- This indicates that Sex is strongly associated with the survival outcome.

```
round(pd.crosstab(titanic['Survived'], titanic['Sex'], normalize='columns') *
      100, 2)

(pd.crosstab(titanic['Survived'], titanic['Sex'], normalize='columns') *
  100).plot(kind='pie', subplots=True, figsize=(10,6),
  autopct='%0.1f%%', labels=['Died', 'Survived']);
```

4.3 Survival vs Embarked

Conclusions

- Passengers embarking at C show a higher survival proportion. This may be related to the passenger-class composition of the group, so we examine Pclass and Embarked together.

```
round(pd.crosstab(titanic['Survived'], titanic['Embarked'],
                 normalize='columns') * 100, 2)
```

Here above we see this that in Embarked C, have survived change is 55.36%, so here may be in Embarked C have most female or may be most passenger travel in Pclass 1.

```
# check for female
round(pd.crosstab(titanic['Sex'], titanic['Embarked'], normalize='columns') *
      100, 2)
```

Here we see this there our female assumption is not correct

```
# check for PClass
round(pd.crosstab(titanic['Pclass'], titanic['Embarked'],
                  normalize='columns') * 100, 2)
```

So we clear see in the Embarked C have most passenger that travel in the Pclass 1.

4.4 Survival vs Age

Conclusions

- Give to priority save the kids means 0 to 10 age.
- Also their are high chances of survived if age is 30 - 40, because in this range mostly passenge travel in PClass 1.
- Younger passengers show a different survival distribution compared with older passengers. However, age is also related to other variables such as passenger class, so the observed relationship should not be interpreted as a causal effect of age alone.

```
sns.kdeplot(titanic[titanic['Survived'] == 1]['Age'], label='Survived',
            fill=True)
sns.kdeplot(titanic[titanic['Survived'] == 0]['Age'], label='Not Survived',
            fill=True)
plt.legend()

print(titanic[titanic['Pclass'] == 1]['Age'].mean())

sns.boxplot(data=titanic, y='Age', hue='Survived');
```

5.Feature Engineering

5.1 On the Fare Column

The Fare column represents the fare paid for a ticket. However, when multiple passengers share the same ticket, the recorded fare can represent the fare associated with the group rather than a single passenger.

To better understand the fare paid per passenger, we create a new feature called Individual Fare.

Individual Fare = Fare / Number of passengers sharing the ticket

```
titanic['SibSp'].value_counts()

titanic[titanic['SibSp'] == 8]

titanic[titanic['Ticket'] == 'CA. 2343']
```

Why combine the training and test datasets?

- The Ticket feature can represent multiple passengers traveling under the same ticket. Therefore, to understand shared-ticket patterns and calculate individual fare correctly, we temporarily combine the training and test datasets.
- This combined dataset is used only for exploratory feature construction and understanding ticket-level information.
- The target variable Survived remains available only for the training observations.

```
df = pd.read_csv('/kaggle/input/datasets/shuofxz/titanic-machine-learning-
from-disaster/test.csv')
df.head()

titanic = pd.concat([titanic, df])
titanic

titanic[titanic['Ticket'] == 'CA. 2343']

titanic['Fare']/(titanic['SibSp'] + titanic['Parch'] + 1)

titanic['Individual Fare'] = titanic['Fare']/(titanic['SibSp'] +
titanic['Parch'] + 1)

titanic.head()
```

Univariate Analysis on Individual Fare

Colclusion

- There are one male and a female their ticket price is 512\$, which is suprisely.

```
titanic['Individual Fare'].describe()

titanic['Individual Fare'].plot(kind='hist', bins=30);

titanic['Individual Fare'].plot(kind='box');

titanic[titanic['Individual Fare'] > 500]
```

Bivariate Analysis on Individual Fare

Colclusion

- People who paid higher fare the survived chance is more.

```
sns.kdeplot(titanic[titanic['Survived'] == 1]['Individual Fare'],
            label='Survived', fill=True)
sns.kdeplot(titanic[titanic['Survived'] == 0]['Individual Fare'], label='Not
            Survived', fill=True)
plt.legend()
```

5.2 On the SibSp and Parch column

SibSp and Parch both provide information about a passenger's family relationships aboard the Titanic.

We combine these variables to create:

Family Size = SibSp + Parch + 1

The additional 1 represents the passenger themselves.

```
# create 'Family_size' col
titanic['Family size'] = titanic['Parch'] + titanic['SibSp'] + 1

titanic.head()

# create 'Family type' col
# 1 = alone
# 2 to 4 = small
# > 5 = large

def family_size(num):
    if num == 1:
        return 'Alone'
    elif num > 1 and num < 5:
        return 'Small'
    else:
        return 'Large'

titanic['Family size'].apply(family_size)

titanic['Family Type'] = titanic['Family size'].apply(family_size)

titanic.head()
```

Univariate Analysis on Family size and Family Type

Colclusion

- Most of the peoples are traveling with 1 to 4 memebers.
- There are a family with traveling with 11 members in Pclass 3.
- There are 60.4% peoples are traving Alone.

```
# Family size
titanic['Family size'].describe()

titanic['Family size'].plot(kind='hist');
```

```
titanic[titanic['Family size'] == 11]

# Family type
titanic['Family Type'].value_counts()

titanic['Family Type'].value_counts().plot(kind='pie', autopct='%0.1f%%',
                                           ylabel= 'Family Type');
```

Bivariate Analysis on Family Type

Colclusion

- In Small family the % of servived people is 57.9%.
- In Alone family the % of servived people is 30.1%.
- In Large family the % of servived people is 16.1%.

```
round(pd.crosstab(titanic['Survived'], titanic['Family Type'],
                  normalize='columns') * 100, 2)

round(pd.crosstab(titanic['Survived'], titanic['Family Type'],
                  normalize='columns') * 100, 2).plot(kind='pie', subplots=True,
                  figsize=(15,6), autopct='%0.1f%%', labels=['Died', 'Survived']));
```

5.3 On the Name column

Extracting Information from Name

The Name column contains structured information that can be extracted into useful features.

We create:

- Surname — family surname
- Title — social/professional title extracted from the passenger's name

Rare titles are grouped into Other to reduce the number of categories.

```
# make Surname col
titanic['Surname'] = titanic['Name'].str.split(',').str.get(0)

# create new col title
titanic['tittle'] =
    titanic['Name'].str.split(',').str.get(1).str.strip().str.split('
').str.get(0)

titanic['tittle'].value_counts()

# replace the unknown title into other
repacement = {
    'Rev.': 'Other',
    'Dr.': 'Other',
    'Col.': 'Other',
    'the': 'Other',
    'Major.': 'Other',
```

```

    'Capt.': 'Other',
    'Jonkheer.': 'Other',
    'Lady.': 'Mrs.',
    'Mme.': 'Mrs.',
    'Ms.': 'Mrs.',
    'Mlle.': 'Mrs.',
    'Dona.': 'Mrs.',
    'Sir.': 'Mr.',
    'Don.': 'Other'
}

titanic['tittle'] = titanic['tittle'].replace(replacement)

```

Univariate Analysis on 'tittle' col

```

titanic['tittle'].value_counts()

titanic['tittle'].value_counts().plot(kind='pie', autopct='%0.01f%%',
    ylabel='');

```

Bivariate Analysis on 'tittle' col

```

temp_df = round(pd.crosstab(titanic['Survived'], titanic['tittle'],
    normalize='columns') * 100)

temp_df

temp_df.plot(kind='pie', subplots=True, figsize=(15,7), layout=(2, 3),
    autopct='%00.1f%%', labels=['Died', 'Survived']));

temp_df

plt.figure(figsize=(8,4))
sns.heatmap(temp_df, annot=True, cmap='winter')

# add % sign
for text in plt.gca().texts:
    text.set_text(text.get_text() + '%')

plt.title('Survived Chance with the tittle')

```

5.4 On the Cabin column

Extracting Deck from Cabin

- The Cabin column contains a large number of missing values.
- Instead of using the complete cabin number directly, we extract the first character of the cabin value, which represents the deck.
- For passengers without a recorded cabin, we assign None.
- This creates a simpler categorical feature called Deck.

```

print(f'{round(titanic['Cabin'].isnull().sum() / len(titanic['Cabin']) *
    100)}%')

```

```

titanic[titanic['Cabin'] == 'G6' ]

titanic[titanic['Surname'] == 'Sandstrom']

titanic['Cabin']

titanic['Cabin'].value_counts()

titanic['Cabin'].fillna('None', inplace=True)

titanic['Cabin'].value_counts()

titanic['Deck'] = titanic['Cabin'].str[0]

titanic['Deck'] = titanic['Deck'].str.replace('N', 'None')

titanic.head()

```

Univariate Analysis On Deck col

Conclusions

- There are 77.5% value are None.

```

titanic['Deck'].value_counts()

titanic['Deck'].value_counts().plot(kind='pie', autopct='%0.1f%%', figsize=(7,7))

```

Bivariate Analysis On Deck col

Conclusions

- There are most survived chance in the Deck B, D, E, F compare to other Deck.
- All Passengers are died that are in Deck T.
- In Deck G 50% passengers are died.

```

pd.crosstab(titanic['Survived'], titanic['Deck'])

round(pd.crosstab(titanic['Survived'], titanic['Deck'], normalize='columns')
      * 100, 2)

plt.figure(figsize=(15,4))
sns.heatmap(round(pd.crosstab(titanic['Survived'], titanic['Deck'],
                           normalize='columns') * 100, 2), annot=True, cmap='winter',
            linewidths=0.5)

# add % sign
for text in plt.gca().texts:
    text.set_text(text.get_text() + '%')
plt.title('Survived Chance In Decks')

titanic.head()

```

6. Key Findings

Based on the exploratory analysis, several important patterns were observed:

Passenger Class

- Survival proportion differs considerably across passenger classes, with first-class passengers showing a higher survival proportion than passengers in lower classes.

Sex

- There is a substantial difference in survival proportions between male and female passengers.

Age

- Age distributions differ between survivors and non-survivors, although age is also related to other passenger characteristics.

Embarked

- Survival proportions vary by embarkation port. Passenger-class composition provides one possible factor associated with this difference.

Family Size

- Passengers belonging to small families show a different survival pattern compared with passengers traveling alone or in large families.

Fare

- Fare is strongly skewed, and the analysis suggests that fare-related variables may contain useful information about passenger survival.

Title

- Passenger titles reveal additional demographic and social information that may be useful for further analysis.

Deck

- Deck information shows differences in survival proportions, although the large number of missing cabin values limits interpretation.

7. Next Steps

The EDA provides several features that may be useful for predictive modeling.

Potential next steps include:

- Handling missing values
- Encoding categorical variables
- Selecting relevant features
- Scaling numerical variables where appropriate
- Building classification models
- Evaluating model performance

```
!pip install runcell
```